

25. WPILib в проекте UBOR

R02 заменяет способ расчёта движения, а не архитектуру управления. Пульт, сетевой протокол, SafetyController и DriveControl остаются теми же. Между сценой Simulator и физической моделью появился интерфейс SimDynamics; его реализация WpilibDynamics вызывает классы WPILib 2026.2.1.

Статус: исходный код интеграции подготовлен. Сборка и выполнение WPILib здесь НЕ проверены: Maven отсутствует, загрузка бинарных зависимостей недоступна. Работоспособность готового WPILib-дистрибутива этим выпуском не заявляется.

Как проходит один цикл

```
TCP/HMAC -> RobotService
-> SafetyController -> DriveControl -> Actuation
-> Simulator -> WpilibDynamics
-> физическое состояние и HAL-датчики
-> Sensors -> следующий цикл контроллера
```

Simulator больше не вычисляет координаты в WPILib-пути. Он получает готовую позу, скорости колёс, путь, скорости механизмов, токи и напряжение. Старый способ расчёта оставлен для регрессионной проверки и всегда называется LEGACY_FIRST_ORDER.

Что осталось от R01

Механика STEP/CAD и электрические схемы не менялись. Аппаратный адаптер Pi4J и код DL4J сохранены. Включение профиля wpilib не собирает профиль full и не требует загружать библиотеки обучения. В собственной программе нет кода C/C++; штатные нативные библиотеки WPILib используются через JNI.

Вариант запуска называется явно. run-simulator теперь вызывает WPILib; run-legacy-simulator вызывает старый JAR. Если WPILib не собран или не загружается JNI, запуск завершается ошибкой. Автоматического перехода к старой физике нет.

25.1. Приводы и батарея

Для дифференциального шасси применяется DifferentialDrivetrainSim. Щётка и приводной ролик транспортёра представлены двумя DCMotorSim. Напряжение на моторе получается из PWM-доли и напряжения шины, а не из прямого назначения скорости колесу.

Параметр	Значение / происхождение
Масса; колея; радиус	30 кг; 0,5 м; 0,1 м — исходные условия R01
Момент инерции по курсу	2,4 кг·м ² — оценка, не измерение CAD-массовой модели
Редукции: колёса / щётка / лента	112,5 / 25 / 38,8 — допущения
Моторы	Собственные наборы DCMotor на 24 В; не NEO/CIM из примера FRC
Батарея	25,6 В; 0,09 Ом; электроника 1 А — допущения
Радиус ролика	0,025 м; скорость ленты = угловая скорость × радиус

Конфигурация находится в config/wpilib-sim.properties. Параметры двигателя относятся к валу ДО редуктора; смещение паспортных характеристик редукторного выхода с коэффициентом редукции удвоит преобразование. Ширина колеи и радиус дополнительно сверяются с константами DriveControl.

Просадка питания

```
I_source = I_logic + sum(max(0, duty * I_winding))
V_loaded = BatterySim.calculateLoadedBatteryVoltage(
    V_open, R_internal, I_source)
```

Ток, который видит датчик мотора, — модуль тока обмотки. Для батареи используется оценка среднего входного тока PWM. Установлено напряжение моделируемой шины 25,6 В: стандартные 12 В из FRC-примеров не подставляются автоматически. Химия, заряд батареи, температура, токовые ограничения драйвера и рекуперативное перенапряжение здесь не моделируются.

25.2. HAL, время и отключение

WpiSensors записывает результат модели в EncoderSim, AnalogInputSim, DIOSim и RoboRioSim, затем считывает его через обычные объекты WPILib. Именно эти показания энкодеров и токов поступают в общий контроллер. Это проверяет программную обратную связь, но не исполняет SPI-протокол LS7366R/MCP3008 и не эмулирует Raspberry Pi.

Виртуальные каналы	Назначение
DIO 0–3	Два энкодера; метры и импульсы согласованы с DriveControl
AI 0	AnalogGyroSim: диагностический угол, не реальный IMU в схеме
AI 1 / AI 2–5	Дальность 1 В/м / токи 0,5 В/А; насыщение 0–5 В
DIO 4–12	NC-защиты, заполнение, разрешение, ошибка драйвера, два луча
RoboRioSim VIn	Симулируемая шина питания, а не питание физического roboRIO

Ось курса в физике положительна против часовой стрелки; AnalogGyro.getAngle использует противоположный знак. Адаптер учитывает это. Гироскоп используется для диагностики и отдельной проверки, а существующий контур DriveControl регулирует скорости по энкодерам.

Частоты и защёлки

Управляющий цикл работает с периодом 20 мс. Физика разбивается на подшаги до 5 мс, независимо от интерфейса пульта. Истечение 200-мс watchdog размыкает виртуальное разрешение; очистка аварийного признака не включает питание. Требуется новое действие Physical ARM. Пауза модели свыше 5 с отклоняется явно, а не маскируется потерей времени.

Обесточить моторы — не то же самое, что мгновенно остановить колёса. Нулевое входное напряжение в DC-модели соответствует электрическому торможению. Для разомкнутой цепи в R02 введена отдельная приближённая модель выбега с пассивным сопротивлением. Её коэффициент НЕ откалиброван; безопасный тормозной путь не подтверждён.

25.3. Сборка и работа с пультом

Требования: JDK 21, Maven 3.9+ и доступ к Maven Central/FRCMaven. В pom зафиксирована WPILib 2026.2.1. Схема сборки использует Java JAR и платформенные архивы *-cpp с JNI/shared-библиотеками. Maven распаковывает библиотеки в target/native; скрипты задают java.library.path и путь поиска зависимых DLL/SO.

```
cd software
mvn -Pwpilib clean verify
```

verify запускает реальные проверки WPILib отдельной JVM. Файлы Java из robot-pi/robot-mi не включаются, пока не выбран профиль full. Для Windows предусмотрен windowsx86-64, для Linux — linuxx86-64, для macOS — osxuniversal; эти платформенные сборки в данной среде не проверены. Linux ARM не является целевой конфигурацией этого настольного запуска.

Windows

```
run-wpilib.cmd --about
run-wpilib.cmd --desktop --nt
run-wpilib.cmd --demo ..\verification\my-demo 90
```

Linux

```
./run-wpilib.sh --about
./run-wpilib.sh --desktop --nt
./run-wpilib.sh --demo ../verification/my-demo 90
```

В пульте сначала нажать Physical ARM (SIM), затем Arm / Disarm и удерживать SPACE. Движение — стрелки/WASD; C — ручной подбор с человеческой меткой. Для записи добавить --record каталог. Это синтетические кадры симулятора, а не фотографии полигона.

Опция --nt публикует Field2d и телеметрию на 127.0.0.1:5810 для Glass. Команды управления по NetworkTables не принимаются. В headless-режиме --server доступны консольные команды arm, estop, restore, jam; сетевой сервер также по умолчанию слушает только loopback.

25.4. Что проверено, а что нет

Проверка в среде подготовки	Результат
Ядро + старый симулятор	50 проверок — PASS
Интерфейс сменной физики	20 проверок с TEST_DOUBLE_NOT_WPILIB — PASS
Конфигурация физических параметров	9 проверок без WPILib — PASS
TCP/HMAC на старой модели	40 обменов и останов после разрыва — PASS
Синтаксис Java	37 файлов, только parse — PASS
Компиляция / исполнение WPILib	НЕ ВЫПОЛНЕНО
WpiVerification	25 проверок подготовлены, НЕ выполнены

Двадцать проверок интерфейса подтверждают, что сцена берёт координаты, скорости и токи из подключённого backend; не пересчитывает их повторно; передаёт останов и заклинивание; закрывает ресурсы ровно один раз. Тестовая модель намеренно имеет отличное от WPILib имя и не подменяет классы edu.wpi.first.*.

Реальная suite WpiVerification должна проверить движение прямо и назад, поворот, знак гироскопа, энкодеры HAL, 25-вольтовую шину, выбег после отключения, вращение щётки/ролика, заклинивание и токи, watchdog, аварийные защёлки, полный сбор трёх предметов и TCP-отключение.

Где смотреть результат

```
robot-wpilib/target/wpilib-report/result.json
robot-wpilib/target/wpilib-report/wpilib-origin.txt
robot-wpilib/target/wpilib-report/oracle-demo/
verification/REPORT_R02_RU.md
```

В начале запуска статус заменяется на RUNNING_NOT_YET_PASSED. При исключении записывается FAIL; только завершение всей suite формирует PASS. Источник JAR фиксируется отдельно. Протоколы r01_archive и r02-core не являются свидетельством выполнения WPILib. Проверка parse не проверяет типы и внешние сигнатуры.

25.5. Границы модели и следующий стенд

Зрение в демонстрации называется SIM_ORACLE_NOT_ML: класс и координаты предмета известны сцене. WPILib не превращает такую демонстрацию в проверку DL4J. Качество детекции, обучение рулению, ложные срабатывания на тени и неизвестных предметах требуют отдельного набора данных и испытаний.

Сбор мусора остаётся геометрическим автоматом: предмет входит в зону подборщика, переходит на ленту и затем в бункер. Скорость продвижения теперь зависит от скорости ролика DCMotorSim. Если механизмы стоят, одной положительной команды недостаточно. Контакт не моделирует пластичность бутылки, изгиб бумаги, наматывание пакета и силы щетины.

Из сцены убрана ошибка R01: проверка collectible() в самой механике запрещала подбор опасного предмета даже при неверном управлении. Теперь материальный объект не имеет такой защиты, и запрет должен обеспечивать контроллер. Отдельная проверка интерфейса подтверждает возможность физического захвата объекта HAZARD при прямой неправильной команде.

Калибровка после появления прототипа

Сначала определить параметры купленных двигателей и редукторов, действительные импульсы энкодеров, массы и моменты инерции. На вывешенных колёсах снять скорость и ток как функции PWM/напряжения; на полу измерить разгон, поворот и свободный выбег с разной загрузкой. Сравнивать временные ряды энкодеров и токов, а не только конечную точку маршрута.

Для этапа сборщика отдельно измерить разгон щётки и ленты, ток при контакте с эталонными предметами и последствия наматывания. Сначала завершить независимую аварийную цепь и физические испытания защиты. Программная симуляция не обосновывает безопасное использование рядом с людьми.

Первичные источники

WPILib: Physics Simulation, Drivetrain Simulation Tutorial, Device Simulation, API DCMotorSim/BatterySim. Исходные сигнатуры и состав зависимостей сверены с репозиторием [wpilibsuite/allwpilib](https://github.com/wpilibsuite/allwpilib), тег v2026.2.1. Полные адреса приведены в [docs/WPILIB_SOURCES.md](#). Новая глава дополняет R01 и не снимает его инженерных ограничений.